

Introduction to Linux (2)

`scp, echo, >, >>, |, curl, micro,`
Editing and running a Python script

Nikolaj Lindberg
`nikolaj.lindberg@ling.su.se`

Part 2 of 2

- ▶ Repetition: The file tree
- ▶ Repetition: Basic commands
- ▶ Redirection of output: `>`, `>>`, `|`
- ▶ Remote copying of files: `curl`, `scp`
- ▶ The `micro` terminal text editor
- ▶ Exercises

Start Using Tab Completion Today! (Also in `linux_commands.pdf`)

After typing a few characters of a command or file name, you can often use *tab completion* to fill in the rest:

```
ls /h
```

```
ls /home/
```

```
ls /home/d
```

```
ls /home/dsv/
```

If there is no unambiguous expansion, **hit** twice for suggestions.

To list all commands beginning `h`:

```
h
```

Start Using Command Line History *Today!*

(Also in `linux_commands.pdf`)

Re-use earlier commands by using the up and down arrows:



Symbols missing from keyboards in the computer lab

(see `linux_commands.pdf`)

\$	option 4			
	option 7			
\	option-shift 7			
[	option 8			
]	option 9			
{	option-shift 8			
}	option-shift 9			
~	option `			(to the right of )

Option  is called *alt* on some keyboards.

Terminal editing shortcuts (see `linux_commands.pdf`)

 **and**  move cursor left and right
 **and**  scroll up or down in command history

 +  jump to beginning of line

 +  jump to end of line

 +  switch place of two characters

 +  clean up terminal window

clear clean up terminal window

 +  search the command history, to reuse earlier commands

 - jump back to previous directory

Exit from commands: `ctrl` + `C`, `ctrl` + `D`, `Q`
([linux_commands.pdf](#))

When you want to abort or exit a command, try one of these:

- `ctrl` + `C` Cancel command (e.g. when you are stuck and bewildered)
- `ctrl` + `D` Stop a command waiting for input (sends EOF, end-of-file).
If the command line is empty: exit the terminal
- `Q` Some interactive commands exit when you type `Q`, *quit*

Does anyone know what a tree is?

The fishing cottage of Charles (Karl) XI, Norra Djurgården
(Built in the 1680's. Not a long walk from Språkstudion)



Photo Holger.Ellgaard - CC BY-SA 3.0

https://sv.wikipedia.org/wiki/Karl_XI:s_fiskarstuga

File tree: yet a rendering (also see previous lecture)

```
/
/home/
/home/dsv/
/home/dsv/kali2818/
/home/dsv/kali2818/public_html/
/home/dsv/kali2818/scripts/
/home/dsv/kali2818/texts/
```

From inside home directory [/home/dsv/kali2818/](#):

```
public_html/
scripts/
texts/
```

Repetition: Basic Linux commands (1)

- ▶ Start terminal: click the search icon (🔍), type `terminal`
- ▶ Log in to server:
`ssh <SU-ID>@triton.dsv.su.se`
- ▶ Current directory:
`pwd` ← Just an arbitrary command to get going
- ▶ Name of computer:
`hostname` ← ...another one
- ▶ Users logged in:
`who` ← ...a command you most likely don't need
- ▶ ...

Repetition: Explain Basic Linux commands (2)

```
cd /home/dsv/kali2818/scripts/
```

```
cd ~kali2818/scripts/
```

Repetition: Explain Basic Linux commands (3)

`cd`

That's right—no arguments.

`cd . .`

Repetition: Explain Basic Linux commands (4)

```
cp ~kali2818/public_html/index.html.tpl .
```

← Dot!

```
cp public_html/index.html.tpl index.html
```

copy and rename the file

Repetition: Explain Basic Linux commands (5)

recursive, in order to copy directory

```
cp -r ~kali2818/scripts/ . ← Dot!
```

```
cp -r scripts/ /tmp/
```

a subdirectory to this, /tmp is
subdirectory

Repetition: Explain Basic Linux commands (6)

```
ls /usr/lib/python3/dist-packages/*.py
```

The asterisk—it means something ↗

Don't be afraid to try it yourself.

Hey! Start using *tab completion*, to spare your fingers and avoid typing mistakes. See `linux_commands.pdf`

Repetition: Explain Basic Linux commands (7)

Most commands take different switches.
What's this?

`ls -a` show hidden files, the files that start with .

Try it in your home directory.
For instance, what might this be

`.bash_history` shows all your commands history

...?

(Have a peek with the help of `less` or maybe `cat`.)

Repetition: Explain Basic Linux commands (8)

```
mv texts/ksl.txt /tmp/
```

```
mv texts/ksl.txt texts/poem.txt
```

rename and moving

Repetition: Explain Basic Linux commands (9)

```
rm writings/memoir/unique_copy.txt
```

```
rm -r magnum_opus/
```

Repetition: Explain Basic Linux commands (10)

Q: How do you recover lost files after issuing a command such as

```
rm -r ragnarök/
```

?

A: You don't!

(Unless maybe if there is some back up system)

Advanced: Learn how to use the `git` version control system

(*Ragnarök* is the end of the world in Norse mythology)

PS: There is a way to lessen the danger of `rm`

By using the `-i` switch, `rm` will prompt you for confirmation:

```
rm -i extremely_important.txt
```

```
rm: remove regular file 'extremely_important.txt'? y
```

(This unfocused user answered 'y', so the file is gone anyway.)

Advanced: You can configure the shell (*bash*) to use an *alias*, to make

```
rm -i
```

how to run *aliases*?

the default when running `rm`. (Go find out for yourself! Search for 'bash alias rm'... or take a look at `~kali2818/.profile`)

Copy between machines: scp (secure copy) (1/2)

The `scp` command is similar to `cp`, but works between different computers.

You need to have a user account on the remote machine.

```
scp user@hostname:<REMOTE FILE> <LOCAL FILE> ← From server
```

or, reversed direction:

```
scp <LOCAL FILE> user@hostname:<REMOTE FILE> ← To server
```

Copy files between machines: scp (secure copy) (2/2)

User kali2818 copies a file, `texts/ksl.txt`, from the remote host `triton.dsv.su.se`.

```
scp kali2818@triton.dsv.su.se:texts/ksl.txt .    ←The dot!  
kali2818@triton.dsv.su.se's password:           ←Enter password  
ksl.txt                                100% 184 1.9KB/s 00:00
```

Copy local file `mtrx.txt` to `texts/` on the remote server:

```
scp mtrx.txt kali2818@triton.dsv.su.se:texts/  
kali2818@triton.dsv.su.se's password:  
mtrx.txt 100% 0 0.0KB/s 00:00
```

Redirecting output to a file: > (echo example)

The echo command **echoes** its argument to the terminal:

```
echo Follow the white rabbit
```



```
Follow the white rabbit
```

(Try it yourself!)

Redirect to a file, using the greater-than symbol:

```
echo Follow the white rabbit > mtrx.txt
```

Inspect the contents of the new file using cat:

```
cat mtrx.txt
```

← My name is **cat**, I print the file to the terminal, meow

```
Follow the white rabbit
```

(Try it yourself!)

The following curl command downloads a file from the internet and prints to the terminal:

```
curl -L https://people.dsv.su.se/~kali2818/ksl.txt
```

```
Över alla mödor
```

file on the internet, it prints
on terminal

```
jag ror.
```

```
Står du på stranden
```

```
och gråter, min bror?
```

```
Gråt inte, bror lilla,
```

```
vänta blott, stilla -
```

```
snart skall du också
```

```
få ro.
```

K. Sivert Lindberg (1933--2024)

You can direct a browser to people.dsv.su.se/~kali2818/ksl.txt, to verify that the file is there.

Add this to the curl command above to redirect output to a file:

```
> ksl_poem.txt
```

```
curl -L people.dsv.su.se/~kali2818/ksl.txt > ksl_poem.txt
```

NB: It is often necessary to use the `-L` switch: `curl -L <URL>`

Overwriting contents of a file: > (echo example)

Repeat the command with a different text, and the previous contents of the file will be replaced:

```
echo Follow the white rabbit > mtrx.txt
echo "Yeah. Sure. I'll go." > mtrx.txt

cat mtrx.txt
Yeah. Sure. I'll go.
```

When the argument to a command includes a special character — in this case ' — you need to (double) quote it.

(Try it yourself! Hit `ctrl` + `C` to cancel if you mess up)

Appending to a file: >>

Create a file as above:

```
echo Follow the white rabbit > mtrx.txt
```

Append a line to the file, using *two* greater-than characters, >>:

```
echo "Yeah. Sure. I'll go." >> mtrx.txt
```

```
cat mtrx.txt
```

(← Inspecting the file using `cat`, purr-purr)

```
Follow the white rabbit
```

```
Yeah. Sure. I'll go.
```

(Wow, two lines)

Redirecting output using > and >> is a general technique, not related to any specific commands.

Appending to a file: >> (Repetition)

```
echo I repeat myself when under stress > kc.txt  
echo I repeat myself when under stress >> kc.txt
```

```
cat kc.txt  
I repeat myself when under stress  
I repeat myself when under stress
```

Redirect to another *command*: | (*pipeline*) 1/2

Redirect a string to `wc` using the **vertical bar |** (option+7):

`wc` can be used to count words.

```
echo How many words? | wc -w  
3
```

← The vertical bar!

Redirect a string to **tr** using **|**:

`tr` translates characters, here from lower to upper:

```
echo I like to shout | tr a-z A-Z  
I LIKE TO SHOUT
```

Redirect to another *command*: | (*pipeline*) 2/2

Look! Three commands in a row:

```
echo live | tr a-z A-Z | rev
```

(rev reverses a line)

EVIL

Inspect result using | less

Sometimes, you want to scroll through the output of a command, using `less` (or `more`):

```
egrep --help | less
```

`egrep` prints selected lines of a file, in this case all lines including the string `whale`:

```
egrep whale texts/moby_dick.txt | less
```

```
echo "I'm gonna party like it's 1777" | tr 7 9
```

When an argument contains special characters, it needs quotes around it. The single-quote ' is a special character (it can be used as string delimiter).

The double quote " is also a special character, along with the vertical bar | , and a few more.

Error messages: Cryptic, but there is a *message* (1/2)

Your probably react with irritation (or worse) to error messages, but if you *read* them, you might get a clue to what went wrong.

Let's say you want to **copy** a file from user kali2818:

```
mv ~kali2818/texts/ksl.txt .
```

but instead you get an error:

```
mv: cannot move '/home/dsv/kali2818/texts/ksl.txt'  
to './ksl.txt': Permission denied
```

What went wrong?

Error messages: Cryptic, but there is a *message* (2/2)

The message

```
mv: cannot move '/home/dsv/kali2818/texts/ksl.txt'  
to './ksl.txt': Permission denied
```

doesn't explicitly explain what went wrong, but it correctly states that you don't have the permission to move the files of user kali2818.

The problem was that you used the wrong command, `mv` instead of `cp`:

```
cp ~kali2818/texts/ksl.txt . (← Success!)
```

It's useful to learn to interpret error messages!

Decrypting an error message: cp example

Try to copy the directory `~kali2818/texts/` to your current directory, `.` (dot):

```
cp ~kali2818/texts/ .  
cp: -r not specified; omitting directory  
    '/home/dsv/kali2818/texts/'
```

Eh...?

The answer is in the error message. Hint: `-r`

The python3 command

You can run a Python script (program) by using the python3 command.

This command runs the Python *interpreter*. The interpreter takes the human readable Python code and compiles it into machine instructions.

This is how you run a script called `wf.py`:

```
python3 wf.py
```

You can find `wf.py` on `triton.dsv.su.se`:
`/home/dsv/kali2818/scripts/wf.py`

A Python script with arguments

It turns out that nothing happens when you run `python3 wf.py` without arguments.

The script `wf.py` needs one or more text files as arguments:

```
python3 wf.py texts/*.txt
```

Copy the following script to your home directory and run it on some text files:

```
cp /home/dsv/kali2818/scripts/wf.py .    (← the period)
```

```
python3 wf.py texts/*.txt
```

(Save the output to a file and inspect the result)

A useful terminal text editor: `micro`

There are many, many, *many* text editors out there.

The `micro` text editor is useful since it's easy to use in the terminal window.

In other words, you can `ssh` to `triton.dsv.su.se` and start programming Python in the terminal!

It's useful to know at least one such light-weight editor, suitable for running in the shell.

(In the long run, you might prefer something more advanced.)

The micro text editor: Step one — set-up

Step one: Run a set-up script to prepare micro for Python files:

1. Log into triton.dsv.su.se using ssh
2. Run the following command:

```
/home/dsv/robe/lis020/bin/setup-lis020
```

sets up the environment

Just type it exactly as above and hit 

3. Log out of triton.dsv.su.se (try  + )
4. Log in again (to activate the new configuration)

The micro text editor: Step two — edit files

You can start the micro text editor with a file name argument:

```
micro memoir.txt  any random name, you name the file  
(start writing...)
```

The expected key-bindings apply:

ctrl	+	S	save
ctrl	+	Q	quit
ctrl	+	Z	undo
↑	+	↓	etc mark text
ctrl	+	C	copy
ctrl	+	V	paste

(Come on, micro, this is too easy)

The micro text editor: Step three — FIND OUT MORE

Here are a few things that you can try to figure out:

- ▶ Editing basics?
- ▶ Working with multiple files?
- ▶ Running a terminal (command line) inside micro?
- ▶ Finding help?

<https://micro-editor.github.io/>

Some micro commands: multiple files

Start with multiple file:

```
micro file1.txt file2.txt file3.txt
```

Switch between open files:

Next: option .

Last: option ,

Split screen:

`ctrl` + `E` then type `vsplit` or `hsplit`

Switch between screens: `ctrl` + `W`

Open a terminal (*shell*) inside micro

ctrl + E then type term

Side note: Files in `public_html` on `triton` become public

Open a browser and go to

```
https://people.dsv.su.se/~kali2818/ksl.txt
```

The file is public, because it resides in `kali2818`'s `public_html/` directory on `triton.dsv.su.se`.

If you put files in *your* `public_html/`, they will become public too.

(Substitute `kali2818` with your own user name, and `ksl.txt` with your own file)

`public_html`: make files public at
`people.dsv.su.se/~<SU-ID>`

- ▶ Put a file in your `public_html/`, e.g. like this:

```
echo Hi everybody! > public_html/hi.txt
```

- ▶ Go to yourself using a browser:

```
https://people.dsv.su.se/~<SU-ID>/ ← Insert your ID
```

- ▶ The file `hi.txt` should be listed and clickable

(Your `public_html/` might become useful later in the course)

index.html: instead of file list

By adding an `index.html` file to `public_html/`, this will be presented instead of the file list.

`https://people.dsv.su.se/~kali2818/` ← Returns `index.html`

Files in `public_html/` are still available, if you know their full path.

`https://people.dsv.su.se/~kali2818/computer_world.txt`

Add and edit index.html

Copy the template file

```
~kali2818/public_html/index.html.tpl
```

to your own `public_html/` and rename it `index.html`

View `https://people.dsv.su.se/~<SU-ID>/` in a browser

Open your `index.html` in `micro` and customise the message as you see fit

(You are welcome to copy my `index.html`, and edit that instead. But mine is a bit silly...)

In Canvas, Introduction to Linux module:

`linux_2_exercises.pdf`

`linux_commands.pdf`

`linux_2_solutions.pdf`

Side note: The Python REPL (Read-Evaluate-Print-Loop)

If you run `python3` without arguments, it starts an interactive session, or REPL:

```
python3 Python 3.11.2 (main, May 2 2024,  
11:59:08) [GCC 12.2.0] on linux  
Type "help", "copyright", "credits" or  
"license" for more  
information.  
>>>
```

In the REPL you can try things out interactively:

```
>>> print(60*60*24)   
86400
```

Quit by  + 

Advanced: Word frequency pipeline

Can you figure out how it works?

Try to break it apart, and test it piece by piece. Try it on some small input text, making it easier to follow.

```
cp -r ~kali2818/texts/ . ← Not part of the pipeline, just copying some data
```

```
cat texts/*.txt|egrep -o '\w+'|sort|uniq -c|sort -rn
```

(What on earth is this...?)



Advanced: How come you don't have to type full paths to commands?

(You don't *need* to know this stuff)

To find a file, you need its path (full or relative), e.g.:

```
/home/dsv/kali2818/texts/deeper_understanding.txt
```

A program (command) is *also* a file, one that you can execute. For example `egrep`:

```
egrep Hello ~kali2818/texts/deeper_understanding.txt
Hello, I know that you've been feeling tired
Hello, I know that you're unhappy
...
```

How come you don't need a path to the `egrep` command?

Advanced: The `which` command

To find out the full path to a command, use `which`, like this:

```
which egrep
```

It will return the path, in this case:

```
/usr/bin/egrep
```

OK, there *is* a path to a command, but how come you can run the command without the full path?

Advanced: The \$PATH environment variable (1/2)

There are a number of *environment variables* (sometimes ENV).

One of them is \$PATH

It holds a list of paths to directories, in which the terminal (the shell) looks for executable programs.

If a program file is in a directory listed in \$PATH, it can be executed without a path.

Advanced: The \$PATH environment variable (2/2)

```
echo $PATH      (← Print the value of $PATH)  
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
```

You can run

egrep

instead of

/usr/bin/egrep

because

/usr/bin/

is in your \$PATH

(You will run a set-up script that adds to your \$PATH, by the way)